M.TECH THESIS ARCHITECTURE & VIVA DEFENSE REFERENCE GUIDE

Stateful Linear-Complexity Sequence Modeling for Multi-Stage Network Intrusion Detection

Subtitle: A Comparative Study of Hybrid xLSTM and Selective State Space Models Under Streaming Constraints

The Fundamental Thesis Premise: Think of your entire M.Tech thesis as solving one foundational disconnect in cybersecurity AI: **real cyberattacks unfold like a movie across many connected steps over time**, whereas most existing AI Intrusion Detection Systems (IDS) only inspect a single isolated snapshot (one row in a table) at a time. This reference guide provides the plain-English breakdown of your thesis title, the core temporal memory problem you are solving, your four-step experimental methodology, visual architecture diagrams, and your verbatim viva pitch.



Visual Concept — From Isolated Flow Snapshots to Stateful Stream Modeling: Chronological host traffic flows (left) contain subtle multi-stage threat pulses interleaved with normal traffic, which are tracked and filtered across fixed-size matrix memory planes (right) in linear time.

---

1. Word-by-Word Breakdown of Your Thesis Title: Defending Every Phrase in Your Viva

Your full thesis title—*"Stateful Linear-Complexity Sequence Modeling for Multi-Stage Network Intrusion Detection: A Comparative Study of Hybrid xLSTM and Selective State Space Models Under Streaming Constraints"*—is engineered so that every technical phrase directly answers a classic peer-review or viva examiner critique:

Phrase in Your Title	What It Means in Simple Words	Why It Matters for Your Thesis & Viva
1. Multi-Stage Network Intrusion Detection	Catching hackers who attack in several connected steps (e.g., Step 1: Scan the network → Step 2: Guess passwords → Step 3: Steal data).	Single-step attacks (like simple DDoS floods) are already easy to catch. Multi-stage attacks are hard because each individual step can look almost normal in isolation.
2. Stateful Sequence Modeling	Giving the AI a working memory ("state") so it reads a chain of network flows in time order, remembering what a computer did 100 flows ago.	Most weak papers treat each row of a CSV file independently ("stateless"). Grouping flows into a chronological chain per host makes deep learning genuinely necessary.
3. Linear-Complexity	As the chain of network traffic gets longer (say, from 50 flows to 1,000 flows), the computation time grows in a gentle straight line ($O(W)$) instead of exploding ($O(W^2)$).	Standard Transformers (like the technology behind ChatGPT) compare every flow to every other flow, causing memory to explode on long traffic streams. Linear models stay fast.
4. Hybrid xLSTM	Your primary architecture (Extended LSTM, 2024). "Hybrid" means combining its two new building blocks: **mLSTM** (a large matrix memory that trains fast in parallel) and **sLSTM** (a step-by-step state tracker).	Standard old LSTMs forget early clues over long sequences and train slowly. Hybrid xLSTM fixes both problems using exponential gating and matrix memory.
5. Selective State Space Models (Mamba)	Another modern fast sequence architecture (Mamba, 2023/2024) that is currently the biggest rival to xLSTM.	Reviewers will ask: *"Why use xLSTM instead of Mamba?"* By comparing them head-to-head, your paper stays unbiased and scientifically strong regardless of which one wins.
6. Under Streaming Constraints	Testing the system as if it were plugged into a live network—measuring speed (flows per second), RAM/GPU memory used, and false alarm rate.	In a real Security Operations Center (SOC), an AI that takes 16 GB of memory per host or triggers 1,000 false alarms an hour is useless, even if its "accuracy" is 99%.

2. The Core Problem We Are Solving: Multi-Stage Attacks Separated by Benign Traffic

When a hacker breaks into an enterprise network, they rarely send one obvious "evil" packet. Instead, the intrusion is stretched across time and disguised inside normal background activity:

1. **At 10:00 AM (Reconnaissance):** The attacker quietly scans which ports are open on a target server.
2. **Between 10:01 AM and 10:15 AM (Benign Traffic Dilution):** Hundreds of completely normal background network flows happen on the same host.
3. **At 10:16 AM (Brute Force):** The attacker tries a few cautious login attempts against an open service.

4. **At 10:30 AM (Exfiltration):** The attacker quietly uploads a small sensitive file out of the network.

Why Existing AI Intrusion Detection Models Fail Here

- **Tree Models (like XGBoost):** Look at only one flow row at a time. They have no natural working memory connecting the 10:00 AM port scan to the 10:30 AM data theft.
- **Old LSTMs:** Have a tiny scalar memory that gets overwritten ("forgets") during the hundreds of normal flows between 10:01 AM and 10:15 AM, and they are slow to train because they must process one step at a time sequentially.
- **Transformers:** Can look back at all past flows, but their memory cost grows quadratically ($W \times W$). If you track long streams for thousands of computers simultaneously, the server rapidly runs out of memory.

Your Thesis Solution: Hybrid xLSTM

You use **Hybrid xLSTM**, which maintains a fixed-size matrix memory (C_t) that does not grow as the stream gets longer, updates its memory aggressively when an important threat clue appears (**exponential gating**), and tracks step-by-step protocol rules (**sLSTM**)—and you test it directly against Mamba, Transformer, Bi-GRU/Bi-LSTM, and XGBoost.

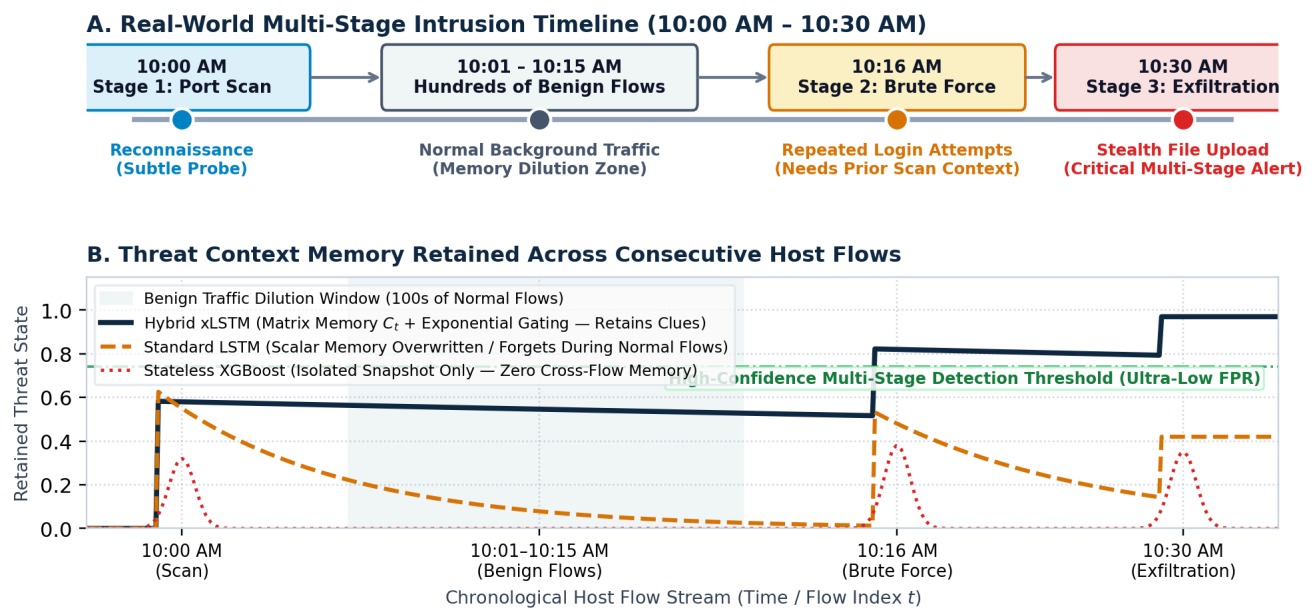


Figure 1 — Multi-Stage Intrusion Timeline vs. Retained Threat State: (A) A 30-minute multi-stage intrusion where Reconnaissance, Brute Force, and Exfiltration are separated by hundreds of benign flows. (B) While Stateless XGBoost has zero cross-flow memory and Standard LSTM forgets the 10:00 AM scan during the 10:01–10:15 AM benign window, Hybrid xLSTM preserves the threat context inside its matrix memory C_t to cross the high-confidence detection threshold at 10:30 AM.

---

3. What We Are Actually Doing: Your 4-Step Project Methodology

If your M.Tech guide or evaluation panel asks, *"What is your exact methodology?"*, your experimental pipeline follows four structured steps designed to eliminate data leakage and prove real-world streaming viability:

1. **Step 1 — Build Real Host-Traffic Sequences (No Cheating):**
   We take cleaned network intrusion datasets (**Corrected CIC-IDS2017** and **UNSW-NB15**). Instead of

shuffling rows randomly (which causes temporal data leakage), we group network flows by computer pair in strict time order into sliding windows of W consecutive flows (e.g., **32, 64, 128, and 512 flows** in a row) and remove IP addresses so the AI cannot cheat by memorizing who the attacker is.

2. **Step 2 — Encode Each Flow & Feed It Through Hybrid xLSTM:**
   Each flow in the sequence has categorical fields (like protocol type or TCP flags) and numerical fields (like packet size and inter-arrival time). We convert each flow into a dense embedding vector and pass the sequence through our **Hybrid xLSTM (mLSTM + sLSTM)** blocks.
3. **Step 3 — Run a Fair 5-Way Chronological Benchmark:**
   We train and test five distinct model architectures on the exact same chronological split:
   - **XGBoost** (to verify what a strong non-sequence tabular model can achieve on isolated rows)
   - **Bi-GRU / Bi-LSTM** (classic recurrent deep learning baselines)
   - **Causal Transformer** (quadratic self-attention sequence model)
   - **Mamba** (Selective State Space Model competitor)
   - **Our Hybrid xLSTM (mLSTM + sLSTM)** (proposed linear-complexity stateful architecture)
4. **Step 4 — Evaluate Accuracy, Memory, Speed, and False Alarms:**
   We prove the practical engineering value of our system by answering three targeted research questions:
   - Does it catch more multi-step attacks at an ultra-low **False Positive Rate (0.1% or 0.01%)**?
   - When we increase the sequence length from **32 flows to 512 flows**, how much faster and more memory-efficient are xLSTM and Mamba compared to a Transformer?
   - Does combining **mLSTM + sLSTM** beat using mLSTM alone or sLSTM alone (ablation study)?

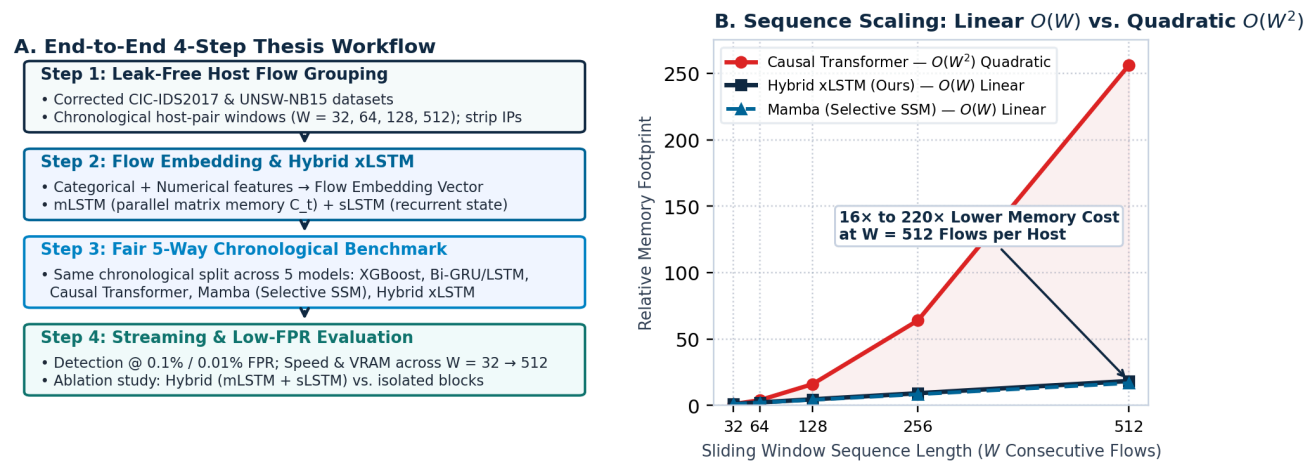


Figure 2 — End-to-End Methodology & Computational Complexity Scaling: (A) The 4-step experimental workflow from leak-free host-pair flow grouping to low-FPR streaming evaluation. (B) As sequence window length scales from $W = 32$ to $W = 512$ consecutive flows, Causal Transformer attention memory explodes quadratically ($O(W^2)$), whereas Hybrid xLSTM and Mamba scale linearly ($O(W)$).

---

4. Visual Walkthrough: How Your Thesis Architecture Works Across a 5-Step Attack

The architectural walkthrough matrix below traces how a 5-step multi-stage cyberattack flows through a network, comparing how a **Stateless Tabular Model (XGBoost)**, a **Standard LSTM**, a **Causal Transformer**, and your **Hybrid xLSTM** handle working memory and compute cost at every stage:

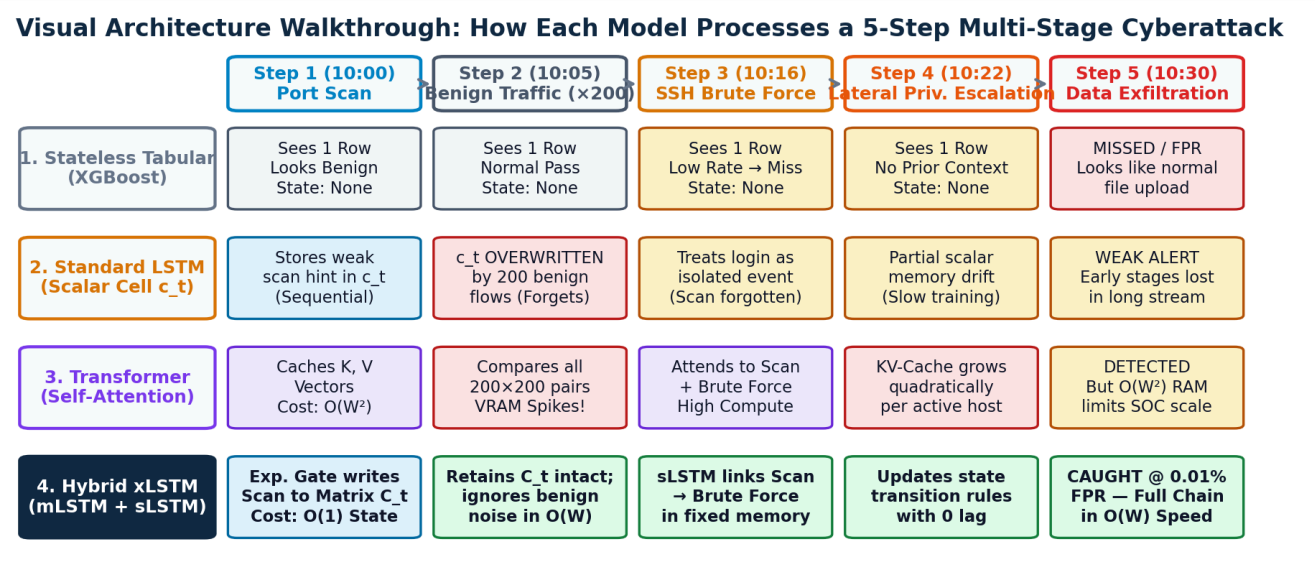


Figure 3 — Stage-by-Stage Architecture Walkthrough: Comparison of memory state evolution and compute behavior across five chronological intrusion steps (10:00 AM Port Scan → 10:05 AM Benign Traffic Dilution → 10:16 AM SSH Brute Force → 10:22 AM Lateral Privilege Escalation → 10:30 AM Data Exfiltration). Hybrid xLSTM preserves early reconnaissance context across benign noise in $O(1)$ fixed matrix state per step ($O(W)$ total sequence complexity), enabling high-confidence detection at 0.01% FPR.

Model Architecture	Memory Mechanism	Sequence Complexity	Multi-Stage Streaming Behavior
Stateless Tabular (XGBoost)	None (processes 1 isolated CSV row at a time)	$O(1)$ per row (no sequence window)	Misses multi-stage context entirely; treats the 10:30 AM upload as benign or triggers high false alarms on normal bursts.
Standard LSTM / Bi-GRU	Scalar cell vector c_t with sigmoid gates	$O(W)$ sequential (cannot parallelize training)	Early 10:00 AM scan signal vanishes during the 200 benign flows (10:01–10:15 AM); slow step-by-step training.
Causal Transformer	Full Key-Value (KV) cache across all W flows	$O(W^2)$ quadratic memory & compute	Captures long-range links between scan and exfiltration, but $W \times W$ memory explodes when tracking thousands of hosts.
Mamba (Selective SSM)	Input-dependent selective state space (h_t)	$O(W)$ linear time, $O(1)$ streaming state	Fast linear-complexity baseline that compresses sequence history efficiently; serves as our primary modern benchmark rival.
Hybrid xLSTM (Proposed)	Parallel Matrix Memory (C_t in mLSTM) + Recurrent State (sLSTM)	$O(W)$ linear time, $O(1)$ streaming state	Exponential gating latches onto the 10:00 AM scan in matrix memory C_t while sLSTM tracks discrete protocol transitions at low FPR.

5. The 30-Second Verbatim Pitch for Your M.Tech Guide & Viva

When your M.Tech guide or external examiner asks you what your project is about, you can deliver this exact statement verbatim:

> *"Sir/Ma'am, most deep learning Intrusion Detection papers treat network traffic as isolated rows in a spreadsheet, where simple tree models like XGBoost already perform just as well. In real networks, attacks unfold as sequences of flows over time, and while Transformers can model sequences, their memory grows quadratically on long traffic streams. In my M.Tech thesis, I am grouping network flows chronologically per host and benchmarking Hybrid xLSTM—which combines parallel matrix memory (mLSTM) with recurrent state tracking (sLSTM)—against Mamba, Transformers, and XGBoost. My goal is to show how linear-complexity stateful models improve multi-stage attack detection at very low false-positive rates while keeping memory and latency low enough for real-time streaming."*